BITS PILANI · WILP · M.Tech (AIML / Data Science)

AIML CZG565 — Machine Learning

Logistic Regression

Contact Session 5 — Module 4 (Part 2)

Turning a score into a chance, why it draws a straight border, how it learns from its mistakes, and how to check honestly if a classifier is any good — all in plain words.

- Intuition
- Analogy
- Worked example
- Solved problem
- Common pitfall
- Key takeaways

Contents

1 Where we are

Last session we built a classifier that does "multiply-and-add" to get a *score*, then bends that score through an **S-curve** to turn it into a *chance between 0 and 1*. That whole thing is **logistic regression**. In symbols:

$$\text{chance of "yes"} = \sigma(\text{score}), \qquad \text{score} = w_0 + w_1 \cdot \text{fact}_1 + w_2 \cdot \text{fact}_2 + \dots$$

where σ ("sigma") is the S-curve from last session, $\sigma(z) = \frac{1}{1+e^{-z}}$ — the little formula that turns any score z into a chance between 0 and 1 (≈ 1 for big positive scores, ≈ 0 for big negative ones, exactly $\frac{1}{2}$ at $z = 0$). If the chance comes out above 0.5 we say "yes," otherwise "no."

Two honest questions are left, and they are this whole session: **how does it draw its border?**, **how does it learn good weights?**, and **how do we know it worked?**

2 Why its border is a straight line

People say "logistic regression is a linear classifier." Let's actually *see* why, with no hand-waving.

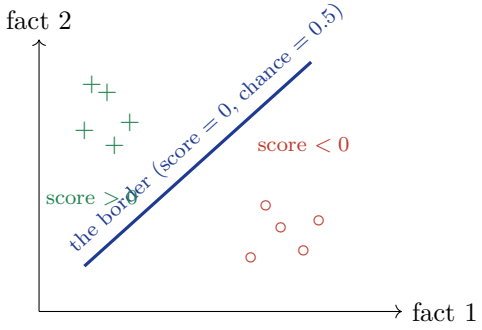
The model says "yes" when the chance is above 0.5. Look back at the S-curve: it crosses 0.5 at exactly one place — where the *score* is 0. So:

$$\text{decide "yes"} \iff \text{chance} > 0.5 \iff \text{score} > 0.$$

The border between yes and no is therefore the set of points where **score** $= 0$. And "score $= 0$" is just

$$w_0 + w_1 \cdot \text{fact}_1 + w_2 \cdot \text{fact}_2 = 0,$$

which is the equation of a *straight line* (a flat sheet if there are more facts). That's the whole reason it's a "linear" classifier — the S-curve only decides *how confident* we are, while the *straight* score-$= 0$ line decides *which side* we're on.



★ The score is a ramp; the border is sea level

Picture the score as a tilted ramp over the floor: it rises as you walk one way, falls the other. The S-curve just colours the ramp — high ground bright ("yes"), low ground dark ("no"). The shoreline where the ramp is exactly at sea level (score 0, chance 0.5) is a perfectly *straight* line. So logistic regression always splits the world with a straight shoreline. (Want a curvy shoreline? Feed in squared facts, exactly as in the last two sessions.)

3 How should it learn? First, how do we score a guess?

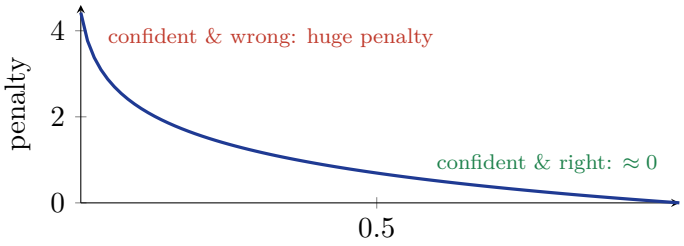
To train the model, we need to tell it how *wrong* a guess is, so it can improve. Here's the gentle way to think about it.

The model outputs a *chance*. Suppose the real answer is "yes."

- If it said "0.99 chance of yes," it was confident and right — it should pay *almost no* penalty.
- If it said "0.5," it was unsure — a *medium* penalty.
- If it said "0.01 chance of yes," it was confident and *wrong* — it should pay a *huge* penalty.

So we want a penalty that is tiny when the model is confidently right and blows up when it's confidently wrong. The penalty that does this is "minus the log of the chance it gave to the true answer":

$$\text{penalty} = -\log(\text{chance it gave to the correct answer}).$$



★ The penalty is "how surprised should you be?"

Think of $-\log(\text{chance})$ as a surprise meter. If you were sure (chance near 1) and you were right, no surprise, no penalty. If you confidently bet against the truth (chance near 0) and it happened anyway, you should be *very* surprised — and the meter shoots up. Training just means: *reduce your total surprise across all the examples.*

Add up this penalty over every example and you get the model's total badness score. It has a name — **log-loss** (or cross-entropy) — but the name matters less than the picture above. In symbols it reads

$$J(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n}\Big[\, y_i \log h_i + (1-y_i)\log(1-h_i)\Big],$$

which looks scary but only says: "for each example, take minus-log of the chance given to the true answer, and average." The little y_i (which is 0 or 1) is just a switch that picks the right half.

! Common Pitfall

Why not reuse the squared-miss score from regression? Because with the S-curve inside, that score becomes a *lumpy* hill full of false bottoms — the ball can roll downhill and still

get stuck in a dent that isn't the real lowest point. The log-loss score, by lovely coincidence, is a smooth single bowl again, so rolling downhill always reaches the best answer.

4 Learning the weights: the same downhill walk

Here's the happy surprise. To improve the weights, we roll downhill on the log-loss bowl — *exactly* the gradient-descent walk from Module 3. And the update rule turns out to look *identical*:

$$\text{new weight} = \text{old weight} - (\text{step size}) \times (\text{average "miss" for that fact}),$$

where now "miss" means *(chance the model gave) minus (the real 0 or 1)*. In symbols,

$$w_j \leftarrow w_j - \frac{\alpha}{n} \sum_{i=1}^{n} (h_i - y_i)\, x_{ij}.$$

★ Key Takeaways

The learning rule is the same sentence you already know: *"nudge each weight against its average miss, by a small step."* You learned it for predicting numbers (regression); it works unchanged for predicting groups (logistic), and later for neural networks too. The *only* thing that changed is that the guess now passes through the S-curve.

∑ Make one prediction

Say training gave us score $= 0.4 + 0.3\,\text{CGPA} - 0.45\,\text{IQ}$. For a candidate with CGPA=5, IQ=6:

$$\text{score} = 0.4 + 1.5 - 2.7 = -0.8, \qquad \text{chance} = \sigma(-0.8) = \frac{1}{1+e^{0.8}} \approx 0.31.$$

The chance of "yes" is 0.31, which is below 0.5, so we predict **no job** — and we're about 69% sure of it. During training, the model would compare this 0.31 to the real answer, get the miss, and nudge the weights.

5 A bonus: the weights actually mean something

Unlike many methods, logistic regression's weights are readable. To see why, meet **odds**. The odds of "yes" are just the chance of yes divided by the chance of no:

$$\text{odds} = \frac{p}{1-p}.$$

If the chance of yes is 0.75, the odds are $\frac{0.75}{0.25} = 3$ — "three to one on," yes is three times as likely as no. Now here's the neat bit. Do a little algebra on the S-curve and the score reappears, all by itself:

$$\log \frac{p}{1-p} = \text{score} = w_0 + w_1 \cdot \text{fact}_1 + w_2 \cdot \text{fact}_2 + \dots$$

Read in words: *the model is a plain straight-line — not in the chance, but in the log of the odds.* So nudging a fact up by 1 *adds* that fact's weight to the log-odds, which *multiplies* the odds by e^{weight}. That single fact makes every weight readable.

∑ Reading a weight

If the CGPA weight is 0.3, then $e^{0.3} \approx 1.35$. So each extra CGPA point multiplies the odds of getting the job by about 1.35 — a 35% bump in the odds, holding everything else fixed. That plain-English readout is why doctors, banks and social scientists love this model.

6 More than two groups

The S-curve handles "yes vs no." For three or more groups (cat / dog / rabbit), two easy tricks:

- **One-vs-rest:** train one yes/no classifier per group ("cat or not," "dog or not," ...) and pick whichever shouts loudest.
- **Softmax:** a direct upgrade of the S-curve that hands out chances to *all* the groups at once, neatly adding up to 100%.

⇝ A panel of specialists vs. one all-rounder

One-vs-rest is asking several specialist doctors, each expert at spotting *one* disease, and trusting whoever is most insistent. Softmax is one all-rounder who weighs every possibility together and reports chances that sum to 100%.

7 Did it work? Don't trust accuracy alone

The obvious score is **accuracy** — the fraction it got right. But accuracy can lie spectacularly.

∑ The 99%-correct useless model

Out of 1000 e-mails, 10 are fraud and 990 are fine. A lazy model that says "fine" to *everything* is right 990 times — 99% accuracy! — yet it catches *zero* frauds. On rare-but-important problems, high accuracy can mean total failure.

So we look at the **four outcomes** of a yes/no classifier, laid out in a little grid called the *confusion matrix*:

	model says yes	**model says no**
really yes	caught it (TP)	missed it (FN)
really no	false alarm (FP)	correctly cleared (TN)

From these we get two honest numbers:

- **Precision** — of all the times it shouted "yes," how often was it right? (Few false alarms ⇒ high precision.)
- **Recall** — of all the real "yes" cases, how many did it catch? (Few misses ⇒ high recall.)

★ You usually can't max both

Lower your bar and you say "yes" more often: you catch more real cases (recall up) but raise more false alarms (precision down). Raise your bar and the reverse happens. So you tune the bar to whichever mistake hurts more. For *cancer screening*, missing a case is terrible ⇒ favour recall. For *spam*, deleting a real e-mail is terrible ⇒ favour precision.

∑ Compute the two numbers

Suppose: caught 40, missed 20, false alarms 10, correctly cleared 130. Then

$$\text{precision} = \frac{40}{40+10} = 0.80, \qquad \text{recall} = \frac{40}{40+20} = 0.667.$$

So when it says "yes" it's right 80% of the time, but it only catches two-thirds of the real cases. Plain accuracy here is $\frac{170}{200} = 85\%$ — rosier than the fuller picture, which is exactly why we look deeper.

Finally, the **ROC curve** and its **AUC** summarise how well the model separates the two groups across *every* possible bar setting — AUC of 1 is perfect, 0.5 is no better than a coin flip. It's the go-to single number for comparing classifiers.

8 The whole story on one page

★ Key Takeaways

- Logistic regression = multiply-and-add a **score**, then bend it through the **S-curve** to get a chance between 0 and 1.
- Its border is **straight** because "chance = 0.5" happens exactly where "score = 0," which is a straight line. (Feed in squared facts for a curvy border.)
- It learns by **rolling downhill** on the *log-loss* score — a penalty that's tiny when confidently right and huge when confidently wrong. The update rule is the same "nudge against the miss" from Module 3.
- The weights are **readable**: each one tells you how a fact tilts the odds.
- For many groups: **one-vs-rest** or **softmax**.
- Don't trust **accuracy** alone on rare problems — use **precision** (few false alarms) and **recall** (few misses), and tune the bar to whichever mistake costs more. **AUC** compares classifiers overall.

Where this comes from: Bishop, *PRML*, Ch. 4; Mitchell, Ch. 6; Andrew Ng's lectures. Course slides M4 (BITS Pilani WILP).